

Lecture 5: Expectations — Transition Dynamics

Computational Methods for Heterogeneous-Agent Macro

Jeffrey Sun

May 15, 2026

University of Toronto

HouseholdStages

HouseholdStages

- An implementation of Stages for building household blocks of heterogeneous-agent macro models.
- Auclert–Bardóczy–Rognlie–Straub **SSJ toolkit** (Python) also has stage implementations.
- Econ-ARK also has a form of Stages.

What the package gives you

- **Stage implementations.** MarkovStage, WealthChangeStage, ConsumptionSavingsStage, LogitChoiceStage, MigrationStage, BorrowingConstraintStage, AssetPriceChangeStage ...
- **Composition.** ○ chains Stages; lift_moments attaches integrals.
- **Inner-solve helpers.** Steady state and transition V , Λ given env (prices and params).
- **Functorial lifts.** lift_jacobian (auto-diff), lift_gpu.
- **SSJ utilities.** Sequence-space Jacobian (covered in L6+).

What the package does *not* do

The library handles **the household block at a given env.** It does *not*:

- Decide your aggregate state.
- Compute prices from $\bar{K}$ (firm block).
- Run outer loops for calibration or market clearing.

Why this split

- Household block: **generic components** across HA models.
- Equilibrium, calibration, transition: **model-specific**.

Steady-state inner-solve helpers

```
# V backward to a fixed point at fixed env.  
solve_vfi_steady_state_given_env!(hh, env)  
  
#  $\Lambda$  forward to stationarity using the policy populated by the backward sweep.  
solve_lambda_steady_state_given_env!(hh)  
  
# Bundle of the two: V then  $\Lambda$ , plus attached moments.  
solve_steady_state_given_env!(hh, env)
```

All three take env **as given**. They iterate the household block but do not update prices or parameters.

Aiyagari Steady State in Stages

Same Model as Before

- **Stage Chain** (time order):

$z_shock \circ income \circ savings.$

- **Firm.** Cobb-Douglas, fixed labor L .
- **Equilibrium.** $R = R^{\text{implied}}(K^{\text{supplied}}(R)).$
- **Outer loop.** Damped **tatonnement** on R .

State Layout

```
layout = StateLayout(  
    StateAxis(:wealth, continuous_grid(0, 100; length=400, spacing=:log)),  
    StateAxis(:z,      [0.6, 1.0, 1.4]),  
)
```

- **wealth** (continuous, 400-pt log grid).
- **z** (productivity; discrete, 3 levels).
- Closures read `cell.wealth`, `cell.z`.

Stage 1: z Shock

```
z_shock = MarkovStage(layout; axis=:z, transition=pΠ._z)
```

Stage 2: Income

```
income = WealthChangeStage(layout; wealth_post=(cell; env) -> env.R * cell.wealth + env.w *  
    cell.z)
```

- Previous example snapped-to-grid. The library function reinterpolates continuously.

Stage 3: Consumption Savings

```
savings = ConsumptionSavingsStage(layout;  $\beta = p\beta.$ , utility=(cell, c; env) -> u_crra(c, Val( $p\sigma.$ )),  
    monotone_search=:divide_conquer)
```

- :divide_conquer: faster $O(N \log N)$ algorithm for when policy is monotone in b .

Compose

```
hh = z_shock ◦ income ◦ savings
```

- ◦ composes Stages in time order.
- The household block is *itself a Stage*: own backward! and forward!.

Define Moments

```
hh = define_moments!(hh; K_supplied=at_end(integrand=:wealth, reduce=sum))
```

- `define_moments!`: Define integrals on Λ . Here $K^{\text{supplied}} = \int b d\Lambda^{\text{end}}$.

Solve Steady State Given R

```
(;K, w) = aiyagari_supply_side(R, p)
env = make_env(hh; R, w)
res = solve_steady_state_given_env!(hh, env)
K_supplied = res.moments.K_supplied
```

- **Line 1.** From R , get firm-side K and wage w .
- **Line 2.** Build env: (R, w) .
- **Line 3.** Find steady state V and Λ .
- **Line 4.** Read off K^{supplied} .

Outer Tatonnement Loop

```
R = R_init
while err >= rtol
    env = make_env(hh; R, w=aiyagari_supply_side(R, p).w)
    (;V, Λ, moments) = solve_steady_state_given_env!(hh, env)
    R_implied = aiyagari_prices(moments.K_supplied, p).R
    err = abs(R_implied - R)
    R += update_speed*(R_implied - R)           # damped tatonnement
end
```

Each pass: $R \rightarrow (K, w) \rightarrow V, \Lambda \rightarrow K^{\text{supplied}} \rightarrow R^{\text{implied}}$, then update R .

Stop when $|R^{\text{implied}} - R| < \text{rtol}$.

Tatonnement on the *price* is the general approach: the household block is a price-taker, and the outer loop searches for the price that clears the market. Tatonnement on $\bar{K}$ happens to work in Aiyagari because asset- and goods-market clearing coincide, but it does not generalize.

Comparative statics

Comparative Statics

Change one parameter and re-solve steady state.

Can think of this as the **long-run** response to a shock.

In the example calibration:

- $A = 1 \rightarrow \bar{K}_{\text{pre}} = [\text{verify}], \bar{R}_{\text{pre}} = [\text{verify}].$
- $A = 1.05 \rightarrow \bar{K}_{\text{post}} = [\text{verify}], \bar{R}_{\text{post}} = [\text{verify}].$

Comparative Statics

Comparative statics tells you **long-run** response to shocks.

Doesn't tell you:

- Speed of transition
- Path of transition
- Welfare impacts of shock on those alive at the time.

For these, we need transition dynamics.

Transition Dynamics

Transition Dynamics

- **Steady state.** A single (K, V, Λ) . Time-invariant.
- **Transition.** A sequence $\{(K_t, V_t, \Lambda_t)\}_{t=1, \dots, T}$. The economy is *moving*.

The MIT Shock

A **permanent, unexpected** shock at $t = 1$.

- $t = 0$: At steady state.
- $t = 1$: Parameter permanently changes. Agents are *surprised*.
- $t \geq 1$: *Perfect foresight* of the entire future path of exogenous and endogenous variables.

Example: TFP Shock

Permanent positive TFP shock at $t = 1$:

$$A_t = 1.05 \quad \text{for all } t \geq 1, \quad A_0 = 1.$$

- The *only* exogenous change.
- $\{R_t, w_t, K_t, V_t, \Lambda_t\}$ change in **response**.
- Set $T = 100$ and set $V_{T+1} = V^{\text{new_steady_state}}$ to close the model.

Solving MIT transition

The problem statement

- **Given:** $\{A_t\}_{t=1}^T$.
- **Find:** $\{R_t, V_t, \Lambda_t\}$ such that
 - $V_{T+1} = V_{\text{ss}^{\text{post}}}$ (terminal).
 - $\Lambda_1 = \Lambda_{\text{ss}^{\text{pre}}}$ (initial).
 - $V_t = \text{backward}(V_{t+1}, \text{env}_t)$.
 - $\Lambda_{t+1} = \text{forward}(\Lambda_t, V_{t+1}, \text{env}_t)$.
 - Markets clear: $R_t = R^{\text{implied}}(K_t^{\text{supplied}})$.

Algorithm

1. Guess path $\{R_t\}$.
2. Iterate V backward.

$$V_{T+1} \leftarrow V_{\text{SS}^{\text{post}}}, \quad V_t = \text{backward}(V_{t+1}, \text{env}_t).$$

3. Simulate Λ forward.

$$\Lambda_1 \leftarrow \Lambda_{\text{SS}^{\text{pre}}}, \quad \Lambda_{t+1} = \text{forward}(\Lambda_t, V_{t+1}, \text{env}_t).$$

4. Update R_{path} .

$$R_t \leftarrow (1 - s) R_t + s R_t^{\text{implied}}, \quad R_t^{\text{implied}} = R^{\text{implied}}(K_t^{\text{supplied}}).$$

Note: Rootfinding

Ultimately, the problem is to find the root of the excess demand function in sequence space:

$$\text{excess_demand}(\{R_t\}) = \vec{0}.$$

For demonstration, we are using the simple and robust method of tatonnement.

Gradient-based methods using the sequence-space Jacobian can be much faster.

The algorithm

```
hh_path = [aiyagari_household(p_new) for _ in 1:T] # one chain per period
R_path = collect(range(pre.R, post.R; length=T)) # initial guess
V_path[T+1] .= post.V; Λ_path[1] .= preΛ. # boundary conditions
while err >= rtol
    env_path = [make_env(hh_path[t]; R=R_path[t], w=aiyagari_supply_side(R_path[t], p_new).w) for t in 1:T]
    for t in T:-1:1 # backward sweep
        copyto!(V_path[t], backward!(hh_path[t], V_path[t+1], env_path[t]))
    end
    for t in 1:T # forward sweep
        copyto!(Λ_path[t+1], forward!(hh_path[t], Λ_path[t]))
        K_S = compute_moments(hh_path[t], Λ_path[t+1], env_path[t]).K_supplied
        R_implied_path[t] = aiyagari_prices(K_S, p_new).R
    end
    err = maximum(abs.(R_implied_path .- R_path))
    R_path .+= update_speed .* (R_implied_path .- R_path) # damped tatonnement
end
```

Implementation

Initial Steady State

We need both:

- $V_{ss^{post}}$: terminal for backward sweep.
- $\Lambda_{ss^{pre}}$: initial for forward sweep.
- Section 2's tatonnement gives both (run once at each end).
- **Setup:** $V_{ss^{post}} \rightarrow V_{path}[T+1]$; $\Lambda_{ss^{pre}} \rightarrow \Lambda_{path}[1]$; initial $\{R_t\}$ a straight line from R_{ss}^{pre} to R_{ss}^{post} .

Backward sweep

```
for t in T:-1:1
    env_t = make_env(hh_path[t]; R=R_path[t], w=aiyagari_supply_side(R_path[t], p_new).w)
    copyto!(V_path[t], backward!(hh_path[t], V_path[t+1], env_t))
end
```


Forward sweep (re-seat kernels!)

```
for t in 1:T
    env_t = make_env(hh_path[t]; R=R_path[t], w=aiyagari_supply_side(R_path[t], p_new).w)
    copyto $\Lambda$ !(_path[t+1], forward!(hh_path[t],  $\Lambda$ _path[t]))
    K_S = compute_moments(hh_path[t],  $\Lambda$ _path[t+1], env_t).K_supplied
    R_implied_path[t] = aiyagari_prices(K_S, p_new).R
end
```

Outer tatonnement update

```
err = maximum(abs.(R_implied_path .- R_path))  
R_path .+= update_speed .* (R_implied_path .- R_path)
```

IRF: capital

Shape.

- ΔA raises R .
- Higher $R \rightarrow$ more savings.
- More saving $\rightarrow$ higher $K \rightarrow$ lower R . Self-correcting.

Lag. Wealth distribution has inertia. Peak in K lags peak in A by ~ 5 periods.

IRF: R_t and w_t

R_t .

- Jumps up on impact (A rises before K moves).
- Decays back toward $R_{ss}^{post} = 1/\beta$ as K accumulates.
- In Aiyagari $R_{ss}^{pre} = R_{ss}^{post}$; K absorbs the permanent shift.

w_t .

- Same pattern, opposite sign on impact.
- Rises with K to a permanently higher w_{ss}^{post} .

Welfare

- Wealthy agents: transient capital gain on R .
- Wage-dependent agents: pick up the w side.

For Aiyagari we get a *distribution* of welfare effects, period by period. Disentangling the two channels is one thing MIT shocks let you do.

Pick T large enough

- T = horizon by which $\bar{K}$ and V have converged to the post-shock steady state, so $V_{T+1} \approx V_{ss}^{\text{post}}$.
- Too small $\rightarrow$ terminal condition contaminates the path.

Rule of thumb. T such that $|\bar{K}_T - \bar{K}_{ss}^{\text{post}}| < 10^{-3}$. We use $T = 200$.